singlecomm

ENGINEERING

Understudy

Synthetic participants that stand in for real people — so an AI agent can prove its own work before a human ever sees it.

> The scenarios that matter are the ones you can't stage alone. Two or more people talking at once. A warm transfer, then a supervisor listening in. A customer who switches language mid-call. A network that dies **during** the call. Each needs people, phones and an afternoon — so it gets tested once, by hand, and then never again.

3 × 2
channels × directions: SingleMeet, voice, webchat

69
committed scenarios, validated on real clusters

274
unit tests, ~97% coverage, enforced as a gate

4
concurrent synthetic callers, distinct agents

What it is

One browser tab is one **understudy**. It injects a synthetic microphone (text-to-speech) and a synthetic camera, then joins as a real party over the real signalling — a real SIP registration, a real conference, a real media room.

It touches only **public seams** — the same browser APIs any customer's browser uses — never application internals. That keeps a test honest, and lets it survive the app being rewritten underneath it.

The platform is the system under test. Understudy drives **every** party — customer and agent both.

Three kinds of party

co-resident
a real product page — the agent console, the customer widget — with media injected into it

standalone
a minimal channel client; parties enter dynamically through public APIs

api-only
no tab at all; supporting counterparties driven purely over the API

The AI play: the model drives it

Most tooling is built for humans and then wrapped for machines. Understudy is the other way round. It ships a **skill**, and a coding agent working a ticket loads it, stands up the exact call the ticket describes, and asserts on what came back — evidence instead of "the diff looks right".

> **An agent that can only read code is guessing. An agent that can place a call and listen to what came back is verifying.**

A scenario is a file you can read

```
id: meet.customer-captions-smoke          tickets: [ACDDEV-2231]
env: { require: [dev, qa], forbid: [prod] }     # the guard — why an agent may run it

parties:
  - id: customer
    channel: { type: meet, direction: inbound, campaign: meet-captions-demo }
    captions: { on: true, language: en }
    media:
      audio: { source: tts, turns: [
        { text: 'Hola, necesito ayuda con mi factura.', lang: es },
        { text: 'Thank you, I will switch to English now.', lang: en } ] }
```

A synthetic customer speaks Spanish; the assertion is that English captions came back from the live translation path. If QA can read it, QA can write it.

Three verbs, and the one that matters

```
understudy run <dir> --env qa --execute
   --headed   watch it happen
   --replicas N   fan out to N concurrent parties

understudy save <dir> --env qa
   freeze a green run's assertion baseline

understudy catalog
   browsable index of every committed scenario
```

The sentence that matters

Run with `--check-baseline` and the run is diffed against the saved baseline and **fails on any regression**. Which means: **a bug repro becomes a committed regression test.** The repro **is** the artefact — not a ticket comment describing one.

Headed mode is the point

The same command with `--headed` puts the browsers on screen. You are not reading a report about a call — you are watching the console an agent would be looking at, while a synthetic customer talks to it. Four people in one room across three languages, captions live, is one command and a repeatable one.

That is the confidence to have **before** something reaches UAT: not "the tests passed", but "I watched it happen, and I can watch it again tomorrow".

Where AI is actually used

Voices	Local text-to-speech by default; a hosted voice when natural audio matters. **Cached by content hash** — text, language, voice, parameters — so a line renders once and every later run is deterministic and free. Reuses an existing vendor credential rather than adding one.
Observed, not generated	Live captions produced by the platform's own speech pipeline are what assertions match against. The AI under test is the product's, not the harness's.
The operator	The coding agent itself, through the shipped skill. This is the primary relationship: **the model is the user.**

Proven end to end

- SingleMeet: multi-party, live cross-language captions
- Webchat: customer bootstrap → routed agent → reply
- Voice inbound: number → IVR → queue → agent
- OmniChannel: one console, all three channels, one run
- Fan-out: concurrent calls across distinct agents
- Agent-assist rule validation · baseline regression guard

What a run leaves behind

The report	Every assertion with its expression, its verdict and what it actually observed — printed, and the input to the baseline.
The baseline	`baseline/baseline.json`, committed beside the scenario, so a regression is a diff rather than a memory.
The evidence	Per-party console logs, screenshots and audio under `runs/`, named after the scenario that produced them.

Guarded by construction

Every scenario declares the environments it may run on, and production is forbidden in the file itself. Secrets are references, never values — nothing an agent drives can reach a cluster it was not given, and no credential passes through the tooling.